Implementación de un sistema de consultas (DML) y vistas

Informe

Realizado por: María Sofia Pabón Pabón

Grupo: G101 Bases de Datos

Docente: Oralia Cortes Grajales

Institución Universitaria Pascual Bravo

15 de noviembre del 2026

1.Diagrama del modelo físico

El modelo físico de la Red social estudiantil pascualina está compuesto por 15 tablas relacionales diseñadas en PostgreSQL 17.

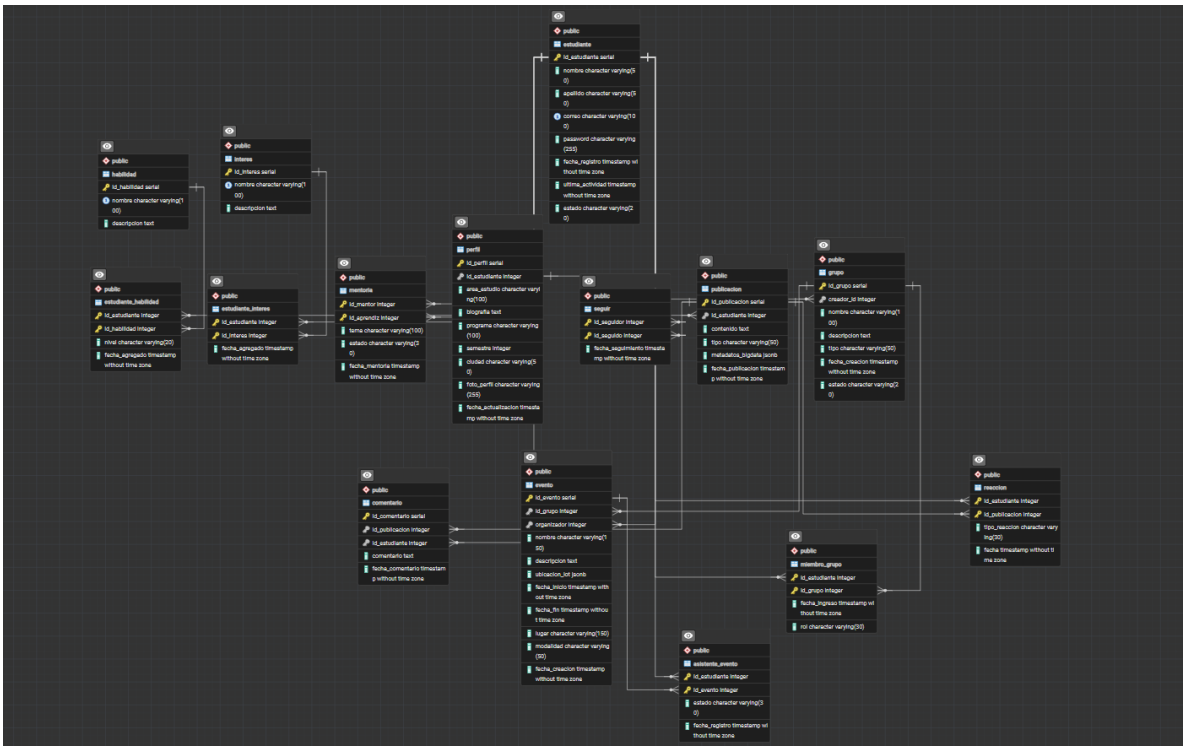
Las tablas principales de las entidades son: estudiante, perfil, publicacion, comentario, grupo, evento, interes, habilidad

Tablas intermedias, relacionales (Muchos a Muchos- M: N): estudiante_habilidad, estudiante_interes, miembro_grupo, asistente_evento, reaccion, seguir, mentoria

Campos NoSQL/ semiestructurados: publicacion.metadatos_bigdata (tipo JSONB): Almacena trazas de dispositivos e IP

evento.ubicacion_iot (tipo JSONB): Registra el aforo máximo permitido e identificadores de sensores IoT

Imagen-Diagrama físico



2.Explicacion del trabajo

Para este trabajo número 5, como en los anteriores el trabajo fue realizado de manera individual, pero siguiendo una metodología de 3 pasos para que la realización del trabajo quedara bien implementada y muy importante organizada:

1. Analice las necesidades clave de la red social estudiantil como lo es buscar tutores, las interacciones de las publicaciones, el aforo de los eventos que se realizan y el seguimiento de la inactividad que se pueda dar en algunos usuarios, para poder realizar las preguntas de negocio
2. Realice una investigación previa para ver que borradores de consultas DML y vistas, se deberían usar y se ajustó las cláusulas JOIN: que básicamente pega las tablas que están conectadas por una llave para que se vea la información en una sola fila organizada, en resumen, cruza las tablas al igual que hice con HAVING que nos Filtra grupos enteros después de contar o promediar
3. Por último, Se construyo la creación del script DML con las 5 consultas y las 3 vistas en PostgreSQL 17 siguiendo las buenas prácticas y construyendo paso a paso lo que es la red social pascualina, trabajada durante todo el semestre, ya desde ahí se empezó con la realización del presente informe y del video

3.Sistema de consultas DML para el modelo de negocio

Consulta 1. Identificación de candidatos a mentores académicos

Pregunta de negocio: ¿Qué estudiante de semestre 4 o superior tienen habilidades en nivel “avanzado” o “experto” para dar mentorías a los demás compañeros que lo requieran?

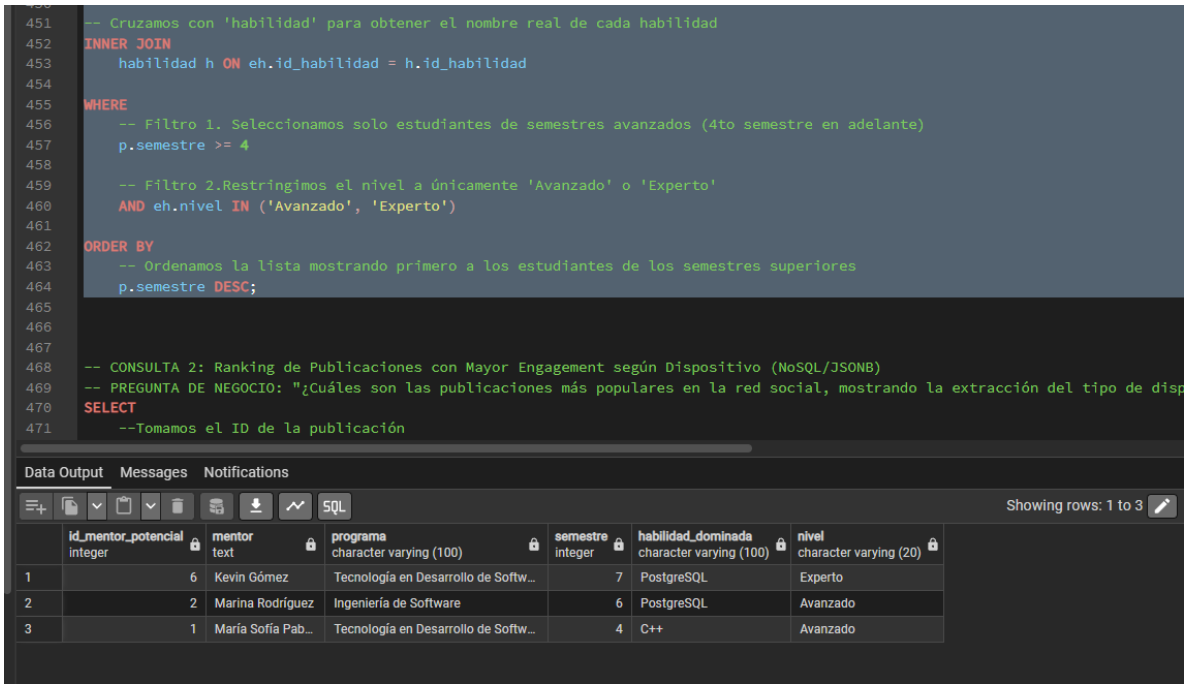
Sentencia SQL

```
SELECT
    e.id_estudiante AS id_mentor_potencial,
    e.nombre || ' ' || e.apellido AS mentor,
    p.programa,
    p.semestre,
    h.nombre AS habilidad_dominada,
    eh.nivel
FROM estudiante e
INNER JOIN perfil p ON e.id_estudiante = p.id_estudiante
```

```
INNER JOIN estudiante_habilidad eh ON e.id_estudiante =
eh.id_estudiante
INNER JOIN habilidad h ON eh.id_habilidad = h.id_habilidad
WHERE p.semestre >= 4
  AND eh.nivel IN ('Avanzado', 'Experto')
ORDER BY p.semestre DESC;
```

Explicación: Esta consulta incluye la unión de 4 tablas (estudiante, perfil, estudiante_habilidad y habilidad) lo que hace es que filtra solo a los estudiantes que están a partir del cuarto semestre o más avanzado y además de eso dominan una materia en nivel avanzado, y ordena los resultados mostrando de primero a los estudiantes de semestres más altos.

Justificación del ALIAS: AS id_mentor_potencial y AS mentor se usan para unir el nombre y apellido en una sola columna limpia que sea fácil de entender a la vista



```
451  -- Cruzamos con 'habilidad' para obtener el nombre real de cada habilidad
452  INNER JOIN
453      habilidad h ON eh.id_habilidad = h.id_habilidad
454
455  WHERE
456      -- Filtro 1. Seleccionamos solo estudiantes de semestres avanzados (4to semestre en adelante)
457      p.semestre >= 4
458
459      -- Filtro 2.Restringimos el nivel a únicamente 'Avanzado' o 'Experto'
460      AND eh.nivel IN ('Avanzado', 'Experto')
461
462  ORDER BY
463      -- Ordenamos la lista mostrando primero a los estudiantes de los semestres superiores
464      p.semestre DESC;
465
466
467
468  -- CONSULTA 2: Ranking de Publicaciones con Mayor Engagement según Dispositivo (NoSQL/JSONB)
469  -- PREGUNTA DE NEGOCIO: "¿Cuáles son las publicaciones más populares en la red social, mostrando la extracción del tipo de disp
470  SELECT
471      --Tomamos el ID de la publicación
```

Data Output | Messages | Notifications

Showing rows: 1 to 3

	id_mentor_potencial integer	mentor text	programa character varying (100)	semestre integer	habilidad_dominada character varying (100)	nivel character varying (20)
1	6	Kevin Gómez	Tecnología en Desarrollo de Softw...	7	PostgreSQL	Experto
2	2	Marina Rodríguez	Ingeniería de Software	6	PostgreSQL	Avanzado
3	1	María Sofía Pab...	Tecnología en Desarrollo de Softw...	4	C++	Avanzado

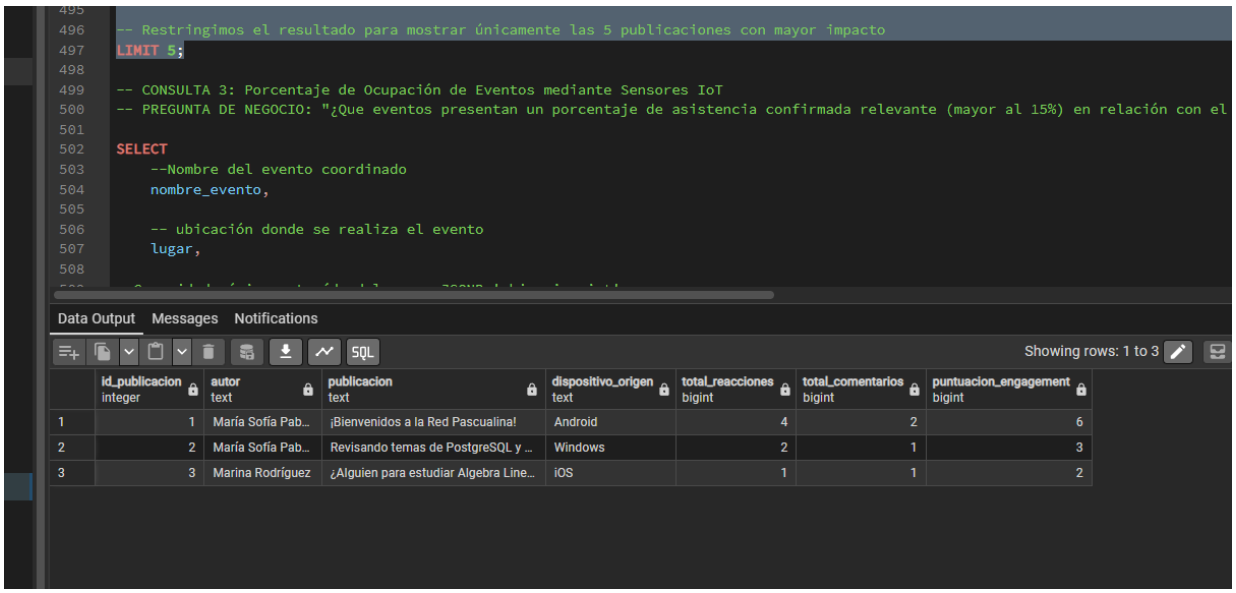
Consulta 2: Ranking de publicaciones con mayor interacción (NoSQL/JSONB) Pregunta de negocio: ¿Cuáles con las publicaciones más populares de la red social y desde que tipo de dispositivo (Android, Windows, iOS) se crearon?

Sentencia SQL

```
SELECT
    id_publicacion,
    autor,
    contenido AS publicacion,
    dispositivo_origen,
    total_reacciones,
    total_comentarios,
    puntuacion_engagement
FROM vw_analitica_publicaciones
WHERE puntuacion_engagement > 1
ORDER BY puntuacion_engagement DESC
LIMIT 5;
```

Explicación: utiliza la vista vw_analitica_publicaciones para obtener el resultado del total de reacciones y comentarios de cada publicación y filtra las que tienen más interacción, las 5 más populares

Justificación de ALIAS: Se usa contenido AS publicación para renombrar la columna original de texto por un nombre más representativo para los administradores



Consulta 3: Porcentaje de Ocupación de Eventos mediante Sensores IoT

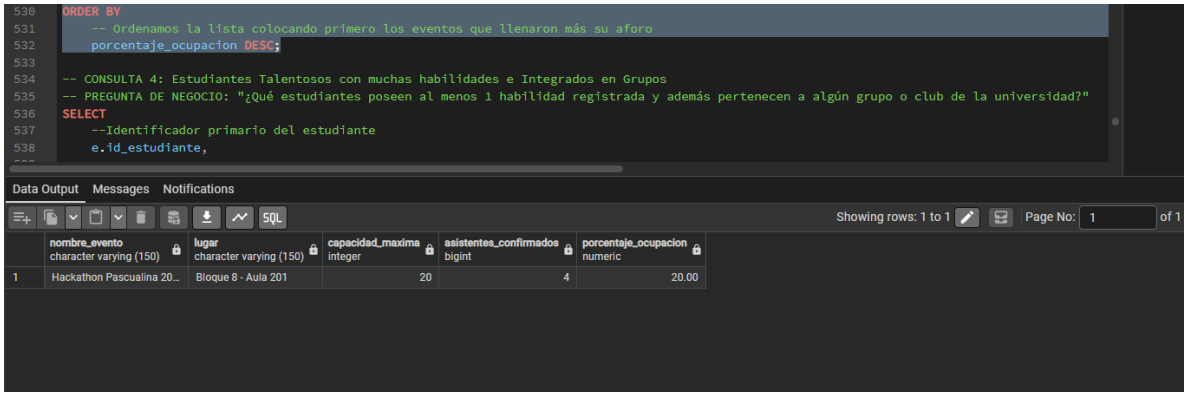
Pregunta de negocio: ¿Que eventos presentan un porcentaje de asistencia confirmada relevante (mayor al 15%) en relación con el aforo máximo registrado por los sensores IoT?

Sentencia SQL

```
SELECT
    nombre_evento,
    lugar,
    capacidad_maxima,
    asistentes_confirmados,
    ROUND((asistentes_confirmados::NUMERIC / capacidad_maxima) *
100, 2) AS porcentaje_ocupacion
FROM vw_metricas_eventos_grupos
WHERE capacidad_maxima > 0
  AND (asistentes_confirmados::NUMERIC / capacidad_maxima) * 100 > 15
ORDER BY porcentaje_ocupacion DESC;
```

Explicación: Toma los datos de la vista vw_metricas_eventos_grupos y divide la cantidad de asistentes confirmados entre la capacidad máxima del sensor IoT y calcula el porcentaje de asistencia del salón o lugar

Justificación de ALIAS: se usa AS porcentaje_ocupacion para darle un nombre al cálculo matemático realizado en dicha consulta



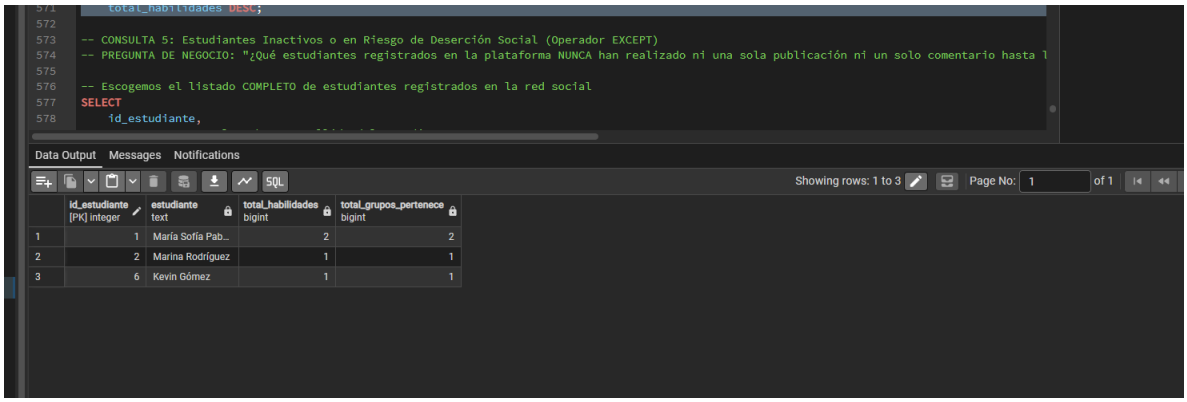
Consulta 4: Estudiantes talentosos integrados en grupos
Pregunta de negocio: ¿Qué estudiantes poseen al menos 1 habilidad registrada y además pertenecen a algún grupo o club de la universidad?

Sentencia SQL

```
SELECT
    e.id_estudiante,
    e.nombre || ' ' || e.apellido AS estudiante,
    COUNT(DISTINCT eh.id_habilidad) AS total_habilidades,
    COUNT(DISTINCT mg.id_grupo) AS total_grupos_pertenece
FROM estudiante e
INNER JOIN estudiante_habilidad eh ON e.id_estudiante =
eh.id_estudiante
INNER JOIN miembro_grupo mg ON e.id_estudiante = mg.id_estudiante
GROUP BY e.id_estudiante, e.nombre, e.apellido
HAVING COUNT(DISTINCT eh.id_habilidad) >= 1
ORDER BY total_habilidades DESC;
```

Explicación: Se une al estudiante con sus habilidades y grupos y usa COUNT (DISTINCT ...) para contar de manera efectiva las habilidades sin que haya posibilidad de que se dupliquen al cruzar las tablas

Justificación de ALIAS: Se asignan total_habilidades y total_grupos_pertenece para representar métricas claras



Consulta 5: Estudiantes inactivos o en riesgo de deserción social
Pregunta de negocio: ¿Qué estudiantes registrados en la plataforma NUNCA han realizado ni una sola publicación ni un solo comentario hasta la fecha?

Sentencia SQL

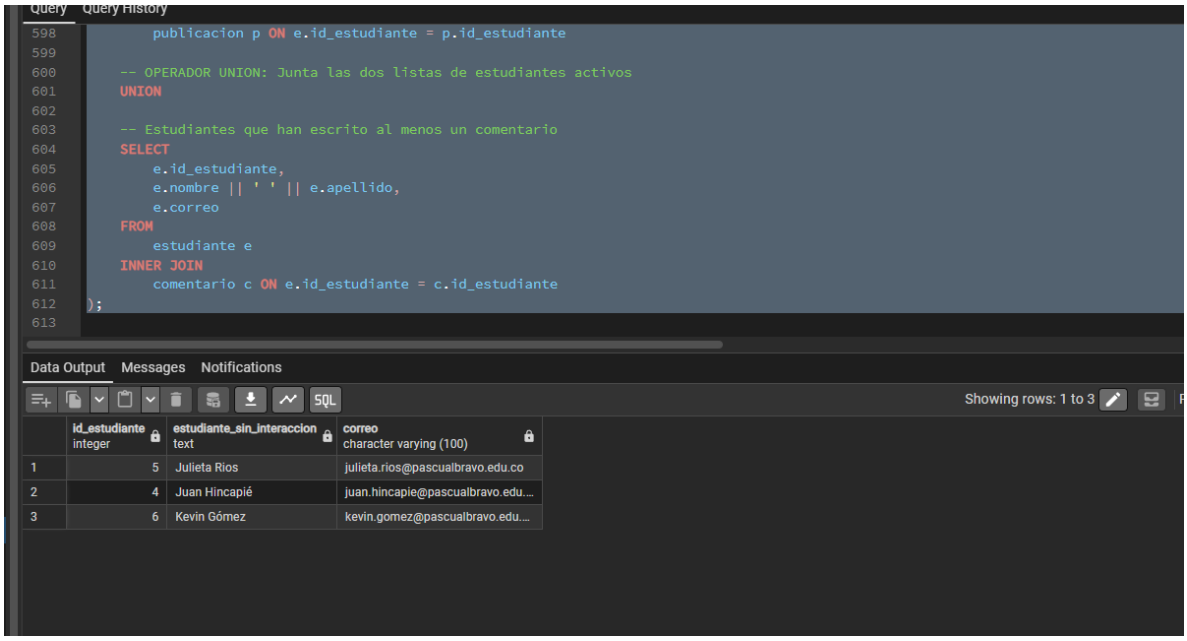
```
SELECT
   id_estudiante,
   nombre || ' ' || apellido AS estudiante_sin_interaccion,
   correo
FROM estudiante
EXCEPT
(
   SELECT e.id_estudiante, e.nombre || ' ' || e.apellido, e.correo
   FROM estudiante e
   INNER JOIN publicacion p ON e.id_estudiante = p.id_estudiante

   UNION

   SELECT e.id_estudiante, e.nombre || ' ' || e.apellido, e.correo
   FROM estudiante e
   INNER JOIN comentario c ON e.id_estudiante = c.id_estudiante
);
```

Explicación: En este caso se aplica la operación de diferencia EXCEPT y toma la lista de todos los estudiantes y les resta aquellos que han creado publicaciones o comentarios y deja únicamente a los estudiantes inactivos

Justificación de ALIAS: Renombra la salida como estudiante_sin_interaccion para que el resultado se enfoque al área de bienestar



4.Diseño e implementación de vistas (VIEWS): Las vistas funcionan como accesos directos o reportes guardados en la base de datos que simplifican de mejor manera las consultas

Vista 1. vw_perfil_estudiantil_completo

El objetivo de esta es reunir en un solo lugar los datos del estudiante, su carrera, su semestre y el conteo de sus habilidades e intereses

Sentencia SQL de creación

```
CREATE OR REPLACE VIEW vw_perfil_estudiantil_completo AS
SELECT
    e.id_estudiante,
    e.nombre || ' ' || e.apellido AS nombre_completo,
    e.correo,
    p.programa,
    p.semestre,
    p.area_estudio,
    COUNT(DISTINCT eh.id_habilidad) AS total_habilidades,
```

COUNT(DISTINCT ei.id_interes) AS total_intereses

FROM estudiante e

LEFT JOIN perfil p ON e.id_estudiante = p.id_estudiante

LEFT JOIN estudiante_habilidad eh ON e.id_estudiante = eh.id_estudiante

LEFT JOIN estudiante_interes ei ON e.id_estudiante = ei.id_estudiante

GROUP BY e.id_estudiante, e.nombre, e.apellido, e.correo, p.programa, p.semestre, p.area_estudio;

Justificación: permite la creación de algoritmos sencillos de recomendación para conectar rápidamente a los estudiantes que apenas inician semestres con tutores o compañeros que conozcan

Sentencia SQL de consulta a la vista:

SELECT * FROM vw_perfil_estudiantil_completo;

```
411
412
413 --Resultados de las 3 vistas
414 SELECT * FROM vw_perfil_estudiantil_completo;
415 SELECT * FROM vw_analitica_publicaciones;
416 SELECT * FROM vw_metricas_eventos_grupos;
417
418 -- 5. LAS 5 CONSULTAS DML DE MODELO DE NEGOCIO
419 -- CONSULTA 1: Identificación de Candidatos a Mentores Académicos
420 -- Pregunta de Negocio: ¿Qué estudiantes de semestre 5 o superior poseen habilidades en nivel "Avanzado" o "Experto" para ofrecer mentorías a lo
421 SELECT
422     -- Identificador del estudiante candidato a ser mentor
423     e.id_estudiante AS id_mentor_potencial,
424
425     -- Se une el nombre y apellido con un espacio para obtener el nombre completo
426     e.nombre || ' ' || e.apellido AS mentor,
427
428     --Traemos el programa académico y el semestre actual del estudiante candidato
429     p.programa,
430     p.semestre,
```

Data Output | Messages | Notifications

Showing rows: 1 to 6 | Page No: 1

	id_estudiante integer	nombre_completo text	correo character varying (100)	programa character varying (100)	semestre integer	area_estudio character varying (100)	total_habilidades bigint	total_intereses bigint
1	1	María Sofía Pabón	sofia.pabon@pascualbravo.edu.co	Tecnología en Desarrollo de Softw...	4	Desarrollo de Software	2	2
2	2	Marina Rodríguez	marina.rodriguez@pascualbravo.edu...	Ingeniería de Software	6	Sistemas	1	1
3	3	Luna Valencia	luna.valencia@pascualbravo.edu.co	Tecnología en Desarrollo de Softw...	2	Inteligencia Artificial	0	1
4	4	Juan Hincapié	juan.hincapie@pascualbravo.edu.co	Ingeniería Electrónica	5	Electrónica	1	0
5	5	Julieta Rios	julieta.rios@pascualbravo.edu.co	[null]	[null]	[null]	0	0
6	6	Kevin Gómez	kevin.gomez@pascualbravo.edu.co	Tecnología en Desarrollo de Softw...	7	Bases de Datos Avanzad...	1	2

Vista 2: vw_analitica_publicaciones

Consolida cada publicación y calcula su número de reacciones, comentarios y extrae el tipo de dispositivo guardado en formato NoSQL(JSONB)

Sentencia SQL de creación

```
CREATE OR REPLACE VIEW vw_analitica_publicaciones AS

SELECT

    p.id_publicacion,

    e.nombre || ' ' || e.apellido AS autor,

    p.contenido,

    p.tipo,

    p.metadatos_bigdata->>'dispositivo' AS dispositivo_origen,

    COUNT(DISTINCT r.id_estudiante) AS total_reacciones,

    COUNT(DISTINCT c.id_comentario) AS total_comentarios,

    (COUNT(DISTINCT r.id_estudiante) + COUNT(DISTINCT c.id_comentario)) AS
puntuacion_engagement

FROM publicacion p

INNER JOIN estudiante e ON p.id_estudiante = e.id_estudiante

LEFT JOIN reaccion r ON p.id_publicacion = r.id_publicacion

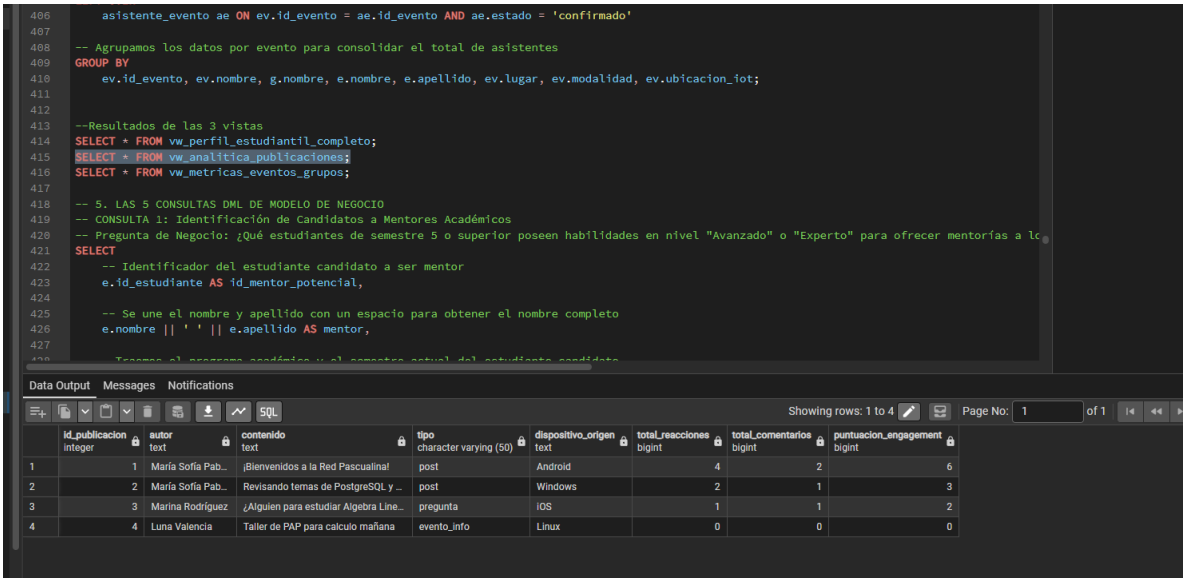
LEFT JOIN comentario c ON p.id_publicacion = c.id_publicacion

GROUP BY p.id_publicacion, e.nombre, e.apellido, p.contenido, p.tipo,
p.metadatos_bigdata;
```

Justificación: Sirve como base para paneles de control que lo que hace es que evalúan que tipo de publicaciones, de las que hacen los estudiantes (académicas, avisos, memes, recursos) generan mayor repercusión y participación

Sentencia SQL de consulta a la vista:

SELECT * FROM vw_analitica_publicaciones;



Vista 3: vw_metricas_eventos_grupos

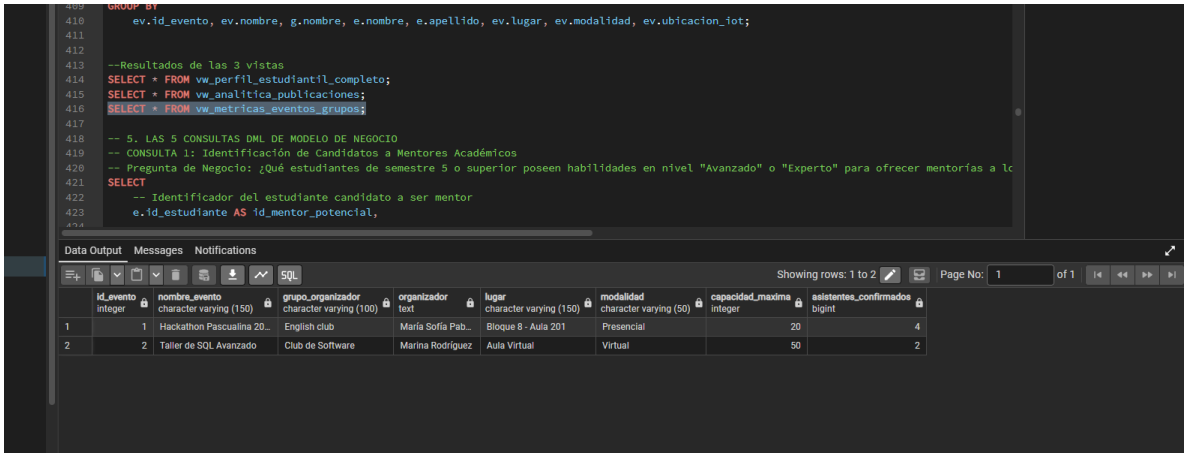
El objetivo es consolidar los eventos realizados por los grupos de estudio convirtiendo el dato de aforo máximo IoT en un numero entero para facilitar los cálculos

Sentencia SQL de Creación:

```
CREATE OR REPLACE VIEW vw_metricas_eventos_grupos AS
SELECT
    ev.id_evento,
    ev.nombre AS nombre_evento,
    g.nombre AS grupo_organizador,
    e.nombre || ' ' || e.apellido AS organizador,
    ev.lugar,
    ev.modalidad,
    (ev.ubicacion_iot->>'aforo_max')::INT AS capacidad_maxima,
    COUNT(ae.id_estudiante) AS asistentes_confirmados
FROM evento ev
LEFT JOIN grupo g ON ev.id_grupo = g.id_grupo
INNER JOIN estudiante e ON ev.organizador = e.id_estudiante
LEFT JOIN asistente_evento ae ON ev.id_evento = ae.id_evento AND
ae.estado = 'confirmado'
GROUP BY ev.id_evento, ev.nombre, g.nombre, e.nombre, e.apellido,
ev.lugar, ev.modalidad, ev.ubicacion_iot;
```

Justificación: Ayuda a la administración de la universidad en el sentido de que ayuda a planificar el uso de aulas físicas según el número de asistentes confirmados

Sentencia SQL de consulta a la vista:
SELECT * FROM vw_metricas_eventos_grupos;



5.Dificultades encontradas

- Simplificación: al juntar varias tablas intermedias a la vez tales como habilidades y grupos, PostgreSQL duplicaba las filas, y lo resolví usando COUNT (DISTINCT ...), lo que obliga a la base de datos a contar cada elemento único sin repetir

- Clausulas especiales: dificultad era cuando intentaba filtrar estudiantes con 1 o más habilidades usando WHERE pero PostgreSQL daba error porque ahí no se permiten funciones de conteo, ya que esa cláusula solo filtra filas individuales antes de agrupar, pero para resultados finales se usa HAVING

- Por último, me pasaba mucho en la ejecución del código me aparecían muchos errores en la sintaxis ya que al ser un proceso tan largo se me pasaban por alto algunos errores

6.Conclusiones y mejoras

Entendí el verdadero valor de SQL y logre transformar muchos registros dispersos como lo son los (likes, comentarios, perfiles entre otros) en respuestas bien implementadas y concretas que le sirven a la universidad, para tener una vista más amplia de la red social pascualina y para tomar decisiones reales, me hace ver que este tipo de trabajos no solo son un ejemplo, es algo que se puede aplicar en la realidad y nos prepara de mejor manera para que mejoremos nuestras habilidades, para ser mejores desarrolladores en un futuro y que tengamos en cuenta todos los procesos que nos llevó hasta acá. Y considero que las ventajas que encontré serian la simplificación de código, y la seguridad, desventajas es que tiene un mayor consumo de recursos si no se materializan las vistas

-Posibles mejoras

Usar Vistas Materializadas (MATERIALIZED VIEW): ya que, en este momento, cada vez que consultamos una vista la base de datos vuelve a hacer todos los cálculos en tiempo real y si la red social llega a tener miles de estudiantes, propongo usar vistas materializadas para que los resultados queden guardados en memoria y se carguen al instante